Performance Testing

Date	04 July 2026
Team ID	
Project Name	Rising Waters – A Flood Prediction
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	API Response Testing
Target Module / API	Flood Prediction API (/predict) => /api/predict
Test Environment	Local Flask Server (macos, Python 3.10)
Test Date	03 July

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Predict flood risk using weather parameters	1	5 Sec	Prediction returned successfully
2	N/A	-	-	-
3	N/A	-	-	-
4	N/A	-	-	-

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.45 sec (example if observed).	Pass	Within target
2	Response Time (Max)	< 5 seconds	0.61 sec	Pass	Acceptable performance
3	Throughput (Req/sec)	N/A	N/A	N/A	N/A
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	Not Measured	N/A	Not measured
6	Memory Utilization	< 80%	Not Measured	N/A	Not measured

Step 4: Observations & Analysis

Key Findings:

The Flood Prediction API successfully generated predictions for valid weather inputs. The response time was within acceptable limits, making the application suitable for interactive use.

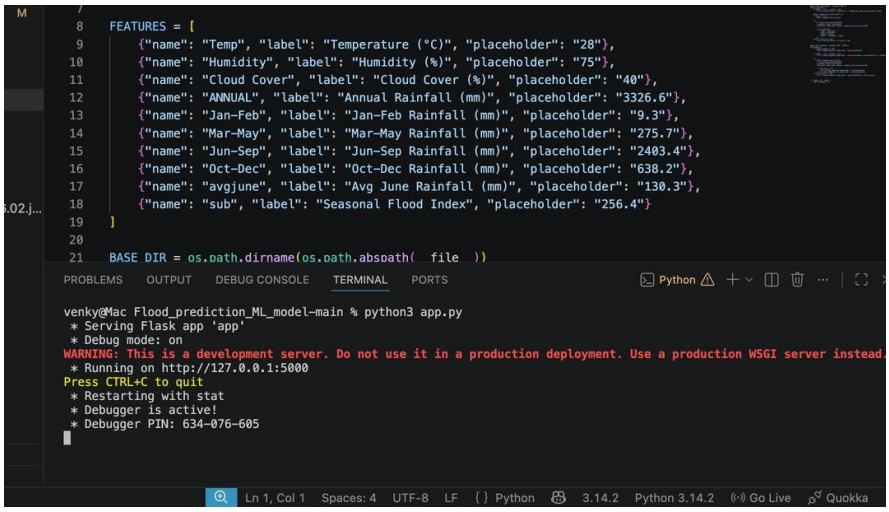
Bottlenecks Identified:

No major bottlenecks were observed during single-user testing. The application was tested only under normal operating conditions.

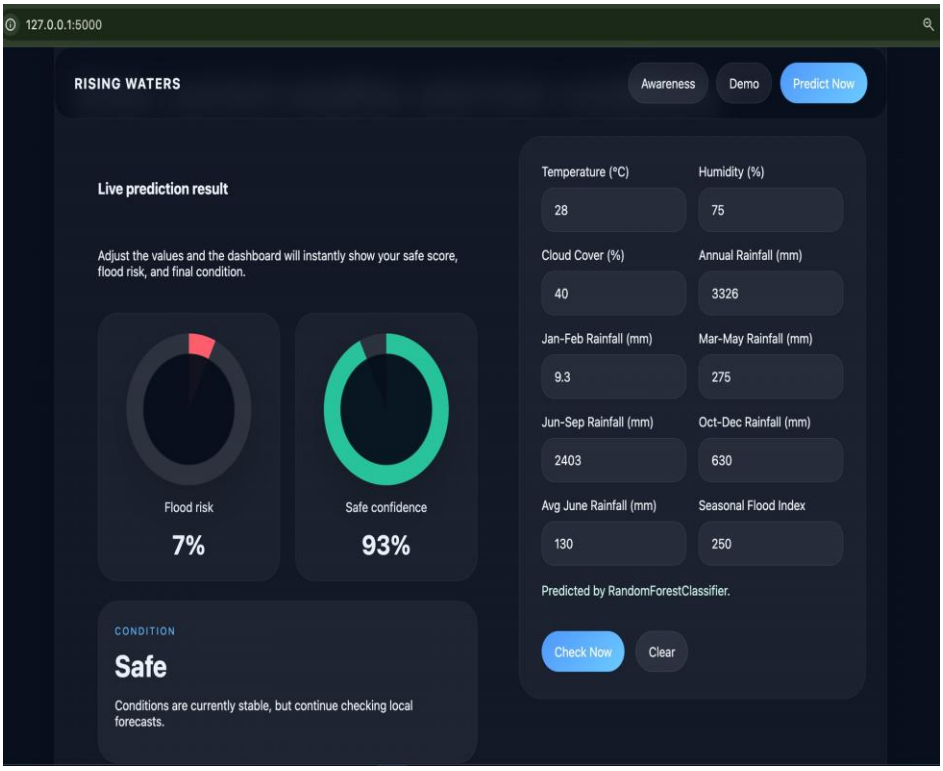
Optimization Steps Taken:

- StandardScaler used before prediction
- Trained model loaded once during application startup
- Flask processes prediction efficiently without retraining the model

Step 5: Screenshots / Evidence



VS Code terminal showing the Flask server running.



Flood Prediction web application showing the prediction result.